

Context Engineering Techniques for AI Agents

Morgan Willis
AWS



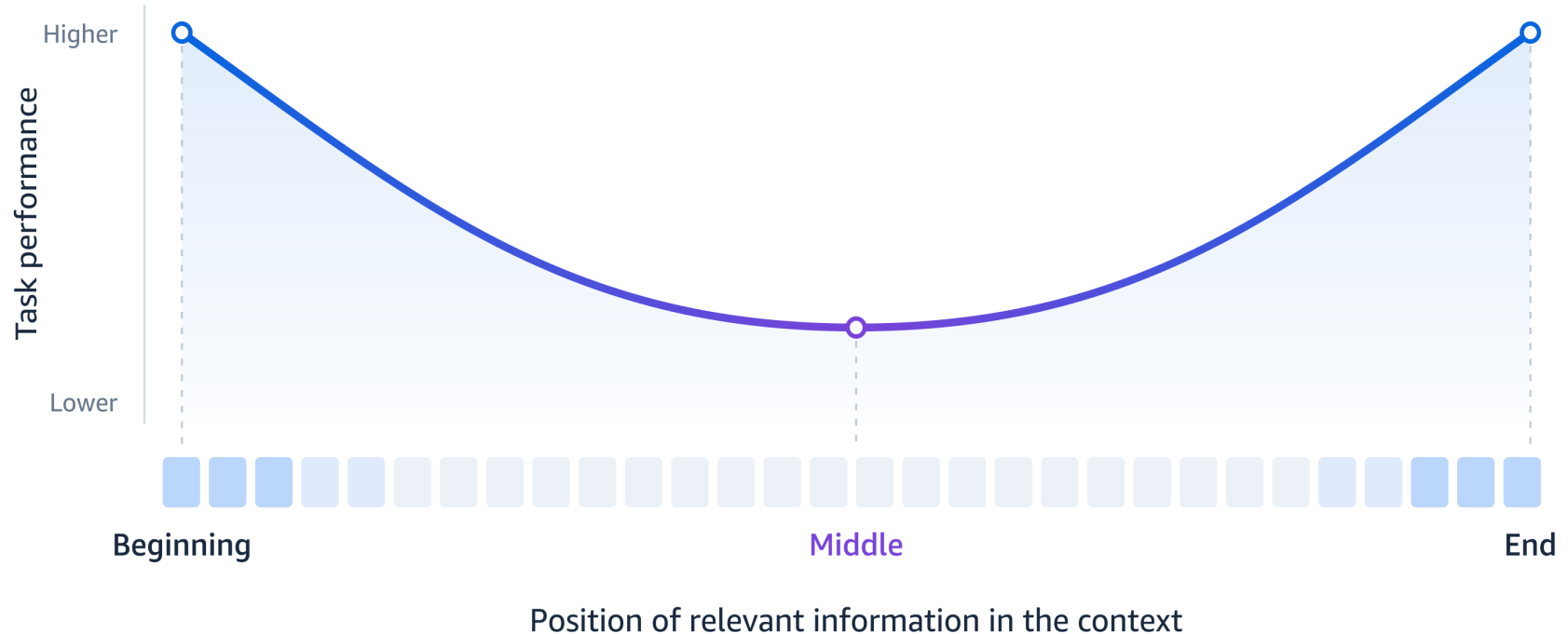
© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Amazon Confidential and Trademark.

4K → 32K → 128K → 1M → 2M tokens

Just add more tokens, that'll fix it

Why It Doesn't Work

Lost in the middle



Context Engineering

Giving the model the information it needs when it needs it.

Curating the context window for optimal outcomes and token consumption.



```
from strands import Agent
from strands.tools.mcp import MCPClient
from strands.vended_tools import file_editor, shell, web_fetch

search = MCPClient.load_servers(
    {"mcpServers": {"exa": {"url": "https://mcp.exa.ai/mcp"}}}
)

agent = Agent(
    system_prompt="You are a coding agent.",
    tools=[file_editor, shell, web_fetch, *search],
)
```



STRANDS

Context Engineering Strategies



Externalize

Externalize to persistent storage.



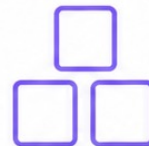
Select

Retrieve only what is relevant.



Compress

Reduce tokens, summarize, compact.



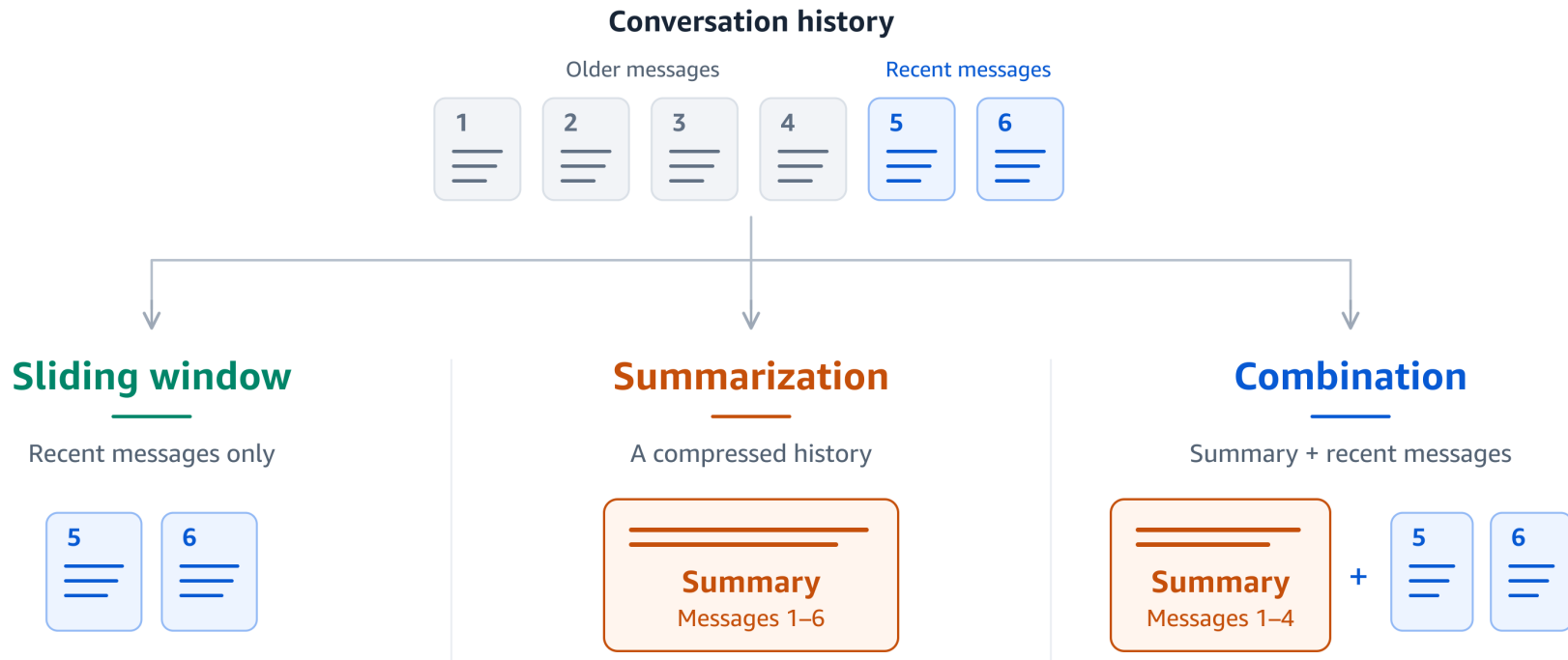
Isolate

Separate context across agents.



Compress

The Starting Point: Conversation Management





Compress

```
from strands import Agent
from strands.experimental.context_manager import (
    Offload,
)

agent = Agent(
    context_manager={
        "strategies": [
            Offload.summarize("*").when(
                utilization=0.85, preserve_recent=4,
            ),
        ],
        "stash": False,
    },
)
```



Tiered Memory



Externalize



Compress

Short-term memory

- Recent conversation history
- Last k-turns always returned
- Sliding window + summarization

Long-term memory

- Extracted from conversation history
- Clean facts > raw conversation
- Scored retrieval

Relational memory

- Entity graph
- Traversal based
- How things connect



Externalize



Select

```
from strands import Agent
from strands.memory import MemoryManager
from bedrock_agentcore.memory.integrations.strands.memorystore import (
    AgentCoreMemoryStore,
)

store = AgentCoreMemoryStore(
    memory_id="YOUR_MEMORY_ID",
    actor_id="demo-user",
    session_id="demo-session",
    namespace="/facts/{actorId}/",
    writable=True,
    extraction=True,
    region_name="us-east-1",
)

agent = Agent(memory_manager=MemoryManager(stores=[store]))
```



STRANDS



Select

Tool-Driven Context Injection

- Agents need more than just conversation history
- Give the agent tools to pull data from knowledge bases (RAG), file systems, MCP servers, and APIs on demand
- Skills use progressive disclosure
- The agent decides what to retrieve and when



Select

```
from strands import Agent, AgentSkills
from strands.vended_tools import file_editor, shell

agent = Agent(
    context_manager="auto",
    plugins=[AgentSkills(skills="./skills")],
    tools=[file_editor, shell],
    system_prompt=(
        "Activate the relevant skill before "
        "starting a specialized task."
    ),
)

agent("Review the API with the code-review skill.")
```



STRANDS

Offload Large Tool Results



Externalize



Select

- Tool results can be verbose and eat up your context window
- Most of it is not relevant to every single turn of the agent loop
- Configure offloading to keep a preview and stash the original
- Let the agent retrieve the full result when it needs more detail



Externalize



Select

```
from strands import Agent
from strands.experimental.context_manager import (
    Offload,
)
from strands.storage import LocalFileStorage

agent = Agent(
    context_manager={
        "strategies": [
            Offload.truncate(
                "tool_results", {"preview_tokens": 500},
            ).when(threshold=2500),
            Offload.summarize("*").when(
                utilization=0.85, preserve_recent=4,
            ),
        ],
        "stash": {
            "storage": LocalFileStorage("./stash"),
        },
    },
    tools=[get_service_logs],
)
```



STRANDS

Context management in one line



Externalize



Select



Compress

- Offload large tool results with on-demand retrieval
- Summarize older messages as context fills



```
from strands import Agent
```

```
agent = Agent(context_manager="auto")
```

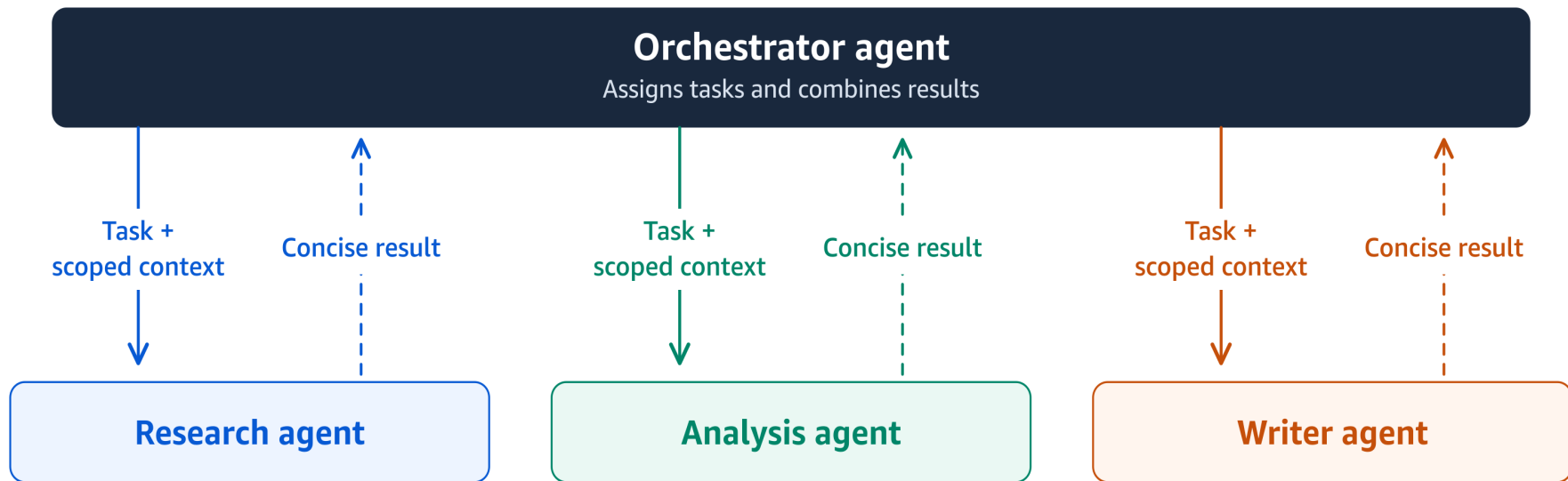


STRANDS

Context-Isolated Subagents



Isolate





Isolate

```
from strands import Agent
```

```
log_analyzer = Agent(  
    name="analyze_logs",  
    description="Investigate logs for a service.",  
    context_manager="auto",  
    tools=[read_logs],  
    system_prompt=(  
        "Return a short diagnosis with key evidence. "  
        "Keep the raw logs in your own context."  
    ),  
)
```

```
orchestrator = Agent(  
    context_manager="auto",  
    tools=[log_analyzer],  
)  
orchestrator("Investigate checkout errors.")
```



STRANDS

Strands harness



Isolate



Externalize



Select



Compress

```
from strands_harness import create_harness
```

```
agent = create_harness(  
    instructions=(  
        "Use the docs-writing and humanize skills. "  
        "Update stale docs and run the examples. "  
        "Report what passed and what you couldn't verify."  
    ),  
)
```

```
agent("Review the latest changes and update the docs.")
```

- Context management and vended tools
- Skills and isolated subagents
- Sessions and long-term memory



STRANDS

Harness benchmarks



Isolate



Externalize



Select



Compress

28%
Lower token cost

- Six benchmarks
- Same Claude or GPT models
- Nearly equal benchmark scores



STRANDS

Personal assistant



Isolate



Externalize



Select



Compress



```
from strands_harness import create_harness
from mem0_strands import Mem0MemoryStore
```

```
memory = Mem0MemoryStore(
    user_id="alex",
    writable=True,
    extraction=True,
)
```

```
agent = create_harness(
    model="openai/gpt-5.4",
    instructions="You are a personal assistant.",
    memory={"stores": [memory]},
    context_manager="agentic",
)
```

```
agent("Help me plan my week around my priorities.")
```

- OpenAI for reasoning
- Mem0 for long-term memory
- Agent chooses what to summarize, drop, or pin



STRANDS

Custom context



Isolate



Externalize



Select



Compress



```
from strands_harness import create_harness
from strands.models.openai import OpenAIModel
from strands.experimental.context_manager import Offload
```

```
summary = Offload.summarize("*", {
    "model": OpenAIModel(model_id="gpt-5-mini"),
    "system_prompt": (
        "Preserve preferences, commitments, dates, "
        "and open tasks. Remove repeated discussion."
    ),
}).when(utilization=0.80, preserve_recent=6)
```

```
agent = create_harness(
    model="openai/gpt-5.4",
    instructions="You are a personal assistant.",
    context_manager={"strategies": [
        Offload.truncate("tool_results").when(threshold=2000),
        summary,
    ]},
)
```

- Separate summary model
- Task-specific summary prompt
- Your thresholds



STRANDS

Thank you!

Try it Yourself

Examples using Strands

